

Technical Analysis Document

Industrial Quality Control - Defect Detection

1. Problem Statement

Immaginiamo di trovarci alla fine di una linea di produzione di piastre metalliche. Ogni pochi secondi un pezzo arriva su un nastro trasportatore, e qualcuno deve guardarlo e decidere: *conforme o da scartare?* Tradizionalmente questo "qualcuno" e' un operatore umano. Il problema e' che il suo occhio si stanca: dopo qualche ora di turno, un **graffio** sottile, un **foro** minuscolo o una **macchia** chiara possono sfuggire - e quel pezzo difettoso finisce dritto in spedizione. Su grandi volumi questo si traduce in resi, danni di immagine e, nei settori safety-critical, addirittura in incidenti.

La Computer Vision nasce esattamente per questo tipo di compiti: *dare alle macchine la capacita' di vedere, interpretare e comprendere il mondo visivo*. Un sistema automatico di **defect detection** non si stanca mai, applica sempre lo stesso criterio e scala con la velocita' della linea. Inquadriamo il problema come una **classificazione binaria** - la domanda piu' semplice della CV: "che cosa c'e' in questa immagine?", con due sole risposte possibili: *pezzo conforme o pezzo difettoso*. A questo aggiungiamo uno stadio di **localizzazione** che, quando il verdetto e' "difettoso", evidenzia dove si trova il difetto. Così il sistema non si limita a dire cosa e se, ma anche *dove* - informazione preziosa per analisi statistiche su quali zone del pezzo sono piu' critiche.

Per affrontare il problema seguiamo una scelta didattica precisa: non costruiamo un solo modello, ne **confrontiamo due**. Da una parte la pipeline **classica**, fatta di descrittori ingegnerizzati a mano (HOG) e di un classificatore lineare (SVM); dall'altra una pipeline **deep learning**, basata su una Convolutional Neural Network pre-allenata (VGG16) con fine-tuning. Il confronto e' interessante per piu' di una ragione: la pipeline classica e' veloce, leggera e interpretabile, ma riesce ancora a tenere il passo con una CNN su un compito di ispezione industriale? Vedremo che la risposta dipende molto dalla natura dei difetti.

2. Methodology

2.1 Il dataset

Una buona regola della Computer Vision e' che, prima ancora dell'algoritmo, conta il dato. Per questo progetto abbiamo scelto un dataset **sintetico**: ogni immagine viene generata programmaticamente con OpenCV. La scelta puo' sembrare controintuitiva - perche' non usare immagini reali? - ma e' motivata da due esigenze didattiche. La prima e' la **riproducibilita'**: chiunque puo' rieseguire il codice e ottenere esattamente lo stesso dataset, senza scaricare archivi esterni. La seconda e' il **controllo**: sappiamo con certezza che tipi di difetti sono presenti, in che proporzione, e a quale intensita'. In un dataset reale tutto questo si scoprirebbe solo a posteriori.

Ogni campione e' un'immagine RGB di **150x150 pixel**. Le piastre "buone" (classe *good*) sono uno sfondo grigio uniforme di intensita' media 180, su cui si aggiunge un leggero rumore gaussiano ($\text{std}=8$) che imita la microtexture metallica. Le piastre "difettose" (classe *defect*) partono dalla stessa base e aggiungono uno fra quattro tipi di difetto, estratti con probabilita' uniforme: un **graffio** tracciato come segmento scuro sottile, un **foro** rappresentato da un cerchio nero pieno, una **macchia** ottenuta sovrapponendo un cerchio semi-trasparente, e una **crepa** simulata con una spezzata di segmenti corti. Lo split e' di **400 immagini per classe in training e 100 per classe in test**, per un totale di 800 campioni di training e 200 di test, perfettamente bilanciati.

Figura 1 — Esempi del dataset sintetico (top: good, bottom: defect)

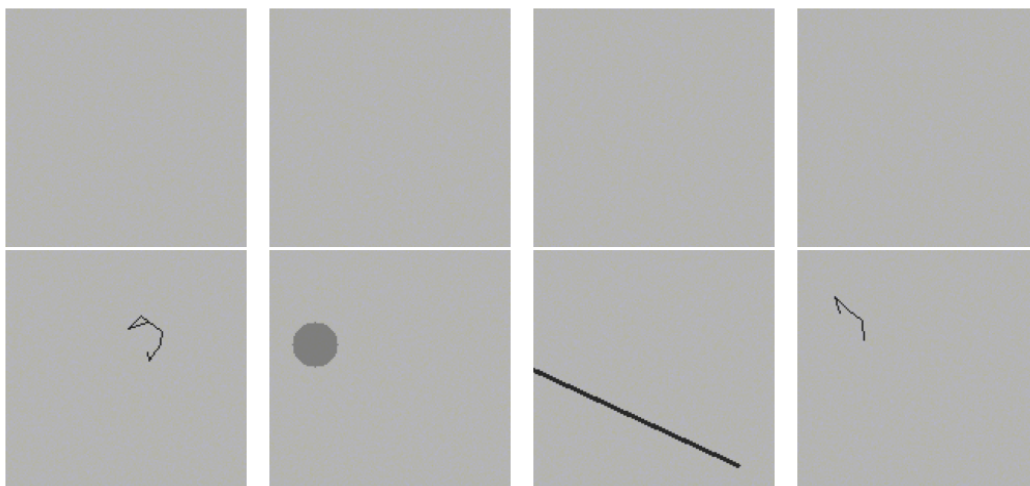


Figura 1 - Esempi del dataset. In alto la classe *good* (sfondi uniformi), in basso la classe *defect* con quattro tipologie diverse di difetto.

2.2 Preprocessing: rendere le immagini pulite

Prima di passare le immagini al classificatore, conviene "pulirle". L'idea e' la stessa che adotteremmo se dovessimo leggere un libro in una stanza poco illuminata: prima accendiamo la luce. La pipeline di preprocessing applica in sequenza tre operazioni classiche, ognuna con un compito preciso.

Si parte dal **bilanciamento del bianco** con l'algoritmo **Gray-World**. L'idea e' semplice: in un'immagine "media", la media complessiva dei tre canali R, G, B dovrebbe essere uguale (grigio neutro). Se un canale ha una media piu' alta, c'e' una dominante cromatica in quella direzione. La correzione consiste nel moltiplicare ciascun canale per il rapporto fra la media globale e la sua media: il canale dominante viene ridotto, quello sottorappresentato viene amplificato.

Segue il **denoising** con un **filtro mediano** di kernel 5x5. A differenza del filtro gaussiano, il mediano preserva meglio i bordi perche' il valore mediano non e' influenzato dagli *outlier*: e' particolarmente efficace contro il rumore *salt-and-pepper*, quei pixel bianchi e neri sparsi casualmente che spesso disturbano le immagini industriali.

Infine, si migliora il contrasto con l'**equalizzazione dell'istogramma**, ma in modo intelligente: si opera *solo* sul canale di luminosit  V dello spazio HSV, lasciando intatti i canali H (tonalit ) e S (saturazione). Se applicassimo l'equalizzazione direttamente sui tre canali BGR separatamente, introdurremmo gravi distorsioni cromatiche; lavorando in HSV, invece, illuminiamo l'immagine senza alterarne i colori.

2.3 Pipeline classica: HOG + SVM

La pipeline classica si basa su un'intuizione che ha dominato la Computer Vision per più di un decennio: se vogliamo riconoscere una forma, dobbiamo descriverla con i suoi **bordi** e le loro **orientazioni**. Il descrittore che fa esattamente questo si chiama **HOG** - *Histograms of Oriented Gradients*, inventato da Naveet Dalal e Bill Triggs nel 2005.

L'algoritmo si svolge in alcuni passi. Prima si converte l'immagine in scala di grigi: HOG non ha bisogno del colore, gli interessa solo come variano le intensità. Si calcolano poi i **gradienti** orizzontale e verticale con un operatore di Sobel: per ogni pixel si ottengono la **magnitudine** del gradiente (quanto è netto il cambio di intensità) e la sua **orientazione** (in che direzione punta). A questo punto l'immagine viene suddivisa in piccole celle quadrate di 8x8 pixel; per ciascuna cella si costruisce un istogramma di 9 bin che coprono gli angoli da 0 a 180 gradi. Il contributo di ogni pixel all'istogramma è pesato dalla sua magnitudine: un pixel che si trova su un bordo netto pesa molto, uno in una zona piatta praticamente nulla.

Segue una fase cruciale di **normalizzazione per blocchi**: le celle vengono raggruppate in blocchi 2x2 sovrapposti e gli istogrammi di ogni blocco vengono normalizzati insieme. È questo passaggio che rende HOG robusto alle variazioni locali di illuminazione: un blocco sovraesposto e uno sottoesposto produrranno lo stesso istogramma normalizzato se la struttura dei gradienti è la stessa. Concatenando tutti gli istogrammi normalizzati otteniamo un unico lungo vettore - il **descrittore HOG**. Con i parametri scelti per il progetto (finestra 128x128, celle 8x8, blocchi 16x16, 9 bin) ogni immagine diventa un vettore di **8100 numeri**.

A questo punto entra in scena il classificatore. Una **Support Vector Machine (SVM)** è un classificatore supervisionato che cerca l'**iperpiano ottimale** per separare i campioni delle due classi massimizzando il margine tra di esse. Tradotto: la SVM disegna un confine fra *good* e *defect* nello spazio delle 8100 dimensioni delle feature HOG, e cerca quello che lascia più "spazio di sicurezza" da entrambi i lati. Per feature HOG, un **kernel lineare** è sufficiente e molto efficiente: le feature HOG sono spesso linearmente separabili nello spazio ad alta dimensione, e aggiungere un kernel non lineare (RBF, polinomiale) rischia solo di causare overfitting senza migliorare la generalizzazione.

2.4 Pipeline deep: VGG16 con fine-tuning

Il salto di paradigma in Computer Vision e' avvenuto nel 2012, quando la rete AlexNet vinse il concorso ImageNet riducendo drasticamente il tasso di errore. Da allora il deep learning - e in particolare le **Reti Neurali Convoluzionali (CNN)** - e' diventato l'approccio dominante. L'idea chiave delle CNN e' sfruttare la **struttura spaziale** dei dati: invece di trattare un'immagine come un vettore piatto, una CNN applica **filtri apprendibili** che scorrono sull'immagine cercando pattern locali - bordi, texture, forme - e poi combinano questi pattern in rappresentazioni sempre piu' astratte man mano che si sale nella rete.

Allenare una CNN da zero richiede pero' milioni di immagini etichettate, qualcosa che il nostro dataset da 800 immagini non ha. La soluzione si chiama **transfer learning**: prendiamo una rete gia' allenata su un dataset enorme (ImageNet, 1.2 milioni di immagini, 1000 classi) e la **riutilizziamo**. L'intuizione e' che gli strati iniziali di qualunque CNN imparano filtri molto generici - rilevatori di bordi, di texture, di gradienti orientati - che sono utili per qualsiasi compito visivo, dal riconoscere gatti al rilevare graffi su una piastra.

Abbiamo scelto **VGG16** come backbone. VGGNet si distingue per la sua semplicita' e uniformita': filtri convoluzionali 3x3 impilati uno sull'altro, max pooling 2x2 fra i blocchi, e una profondita' di 16 strati con pesi addestrabili. Rimuoviamo la testa di classificazione originale di VGG16 (quella che distingueva 1000 classi di ImageNet) e ne montiamo una nostra: un **Flatten**, uno strato **Dense con 256 neuroni e attivazione ReLU**, un **Dropout(0.5)** per regolarizzare, e uno strato finale **Dense con un neurone e attivazione sigmoid**, che produce una probabilita' fra 0 e 1.

Il training si svolge in **due fasi**. Nella prima il backbone VGG16 e' completamente congelato: solo i nuovi strati Dense imparano. Così' diamo alla testa di classificazione il tempo di stabilizzarsi senza che gli aggiornamenti distruggano i pesi pre-allenati. Nella seconda fase si **sbloccano gli ultimi 4 strati convoluzionali** di VGG16 e si continua l'addestramento con un learning rate molto piu' basso ($1e-5$, centomila volte piu' piccolo di quello iniziale). Questo permette agli strati piu' profondi della rete - quelli che codificano feature specifiche del dominio - di adattarsi alle texture metalliche e ai pattern dei difetti, mentre i layer superficiali restano congelati perche' i loro rilevatori di bordi e texture sono gia' perfetti per qualunque immagine.

Per spremere il massimo dal nostro dataset limitato, durante il training applichiamo **data augmentation**: rotazioni fino a 40 gradi, traslazioni del 20%, zoom, shear, flip orizzontale. In questo modo da ogni immagine originale ne otteniamo decine di varianti, e la rete impara a riconoscere il difetto indipendentemente da come e' orientata la piastra. Sul test set, invece, applichiamo *solo* la normalizzazione $rescale=1./255$: l'aumento in valutazione renderebbe le metriche non riproducibili.

2.5 Post-processing: trovare dove si nasconde il difetto

Una volta che il classificatore ha deciso "difettoso", ci serve ancora un'informazione: *dove* nell'immagine si trova esattamente il difetto. Per ottenerla usiamo una pipeline classica di image processing che non richiede annotazioni a livello pixel in fase di training - un grosso vantaggio pratico.

Il primo passo e' la **binarizzazione di Otsu**. L'idea di Otsu, inventata da Nobuyuki Otsu nel 1979, e' brillante: invece di chiedere all'utente di scegliere a mano una soglia, la calcoliamo automaticamente cercando il valore che **massimizza la varianza tra le classi** foreground e background (equivalentemente, minimizza la varianza intra-classe). L'assunzione di Otsu e' che l'istogramma dell'immagine sia **bimodale**: due picchi distinti, uno per lo sfondo chiaro (la piastra) e uno per il foreground scuro (il difetto). Nel nostro caso questa condizione e' perfettamente soddisfatta.

Il risultato di Otsu e' una maschera binaria, ma cruda: ci possono essere pixel bianchi isolati dovuti al rumore di sogliaatura, oppure piccoli buchi neri all'interno dei difetti. Per pulirla applichiamo due operazioni morfologiche con kernel 3x3. La prima e' un **opening** (erosione seguita da dilatazione), che rimuove le componenti isolate piccole. La seconda e' un **closing** (dilatazione seguita da erosione) ripetuto due volte, che riempie i buchi interni e leviga i contorni. Sulla maschera finale applichiamo **connectedComponentsWithStats**, che etichetta tutte le regioni connesse e per ognuna fornisce direttamente il bounding box (x, y, larghezza, altezza) e l'area in pixel. Le regioni con area inferiore a 20 pixel vengono scartate come *rumore residuo*, le altre diventano i bounding box dei difetti.

Figura 4 — Localizzazione del difetto via Otsu + morfologia + connectedComponents

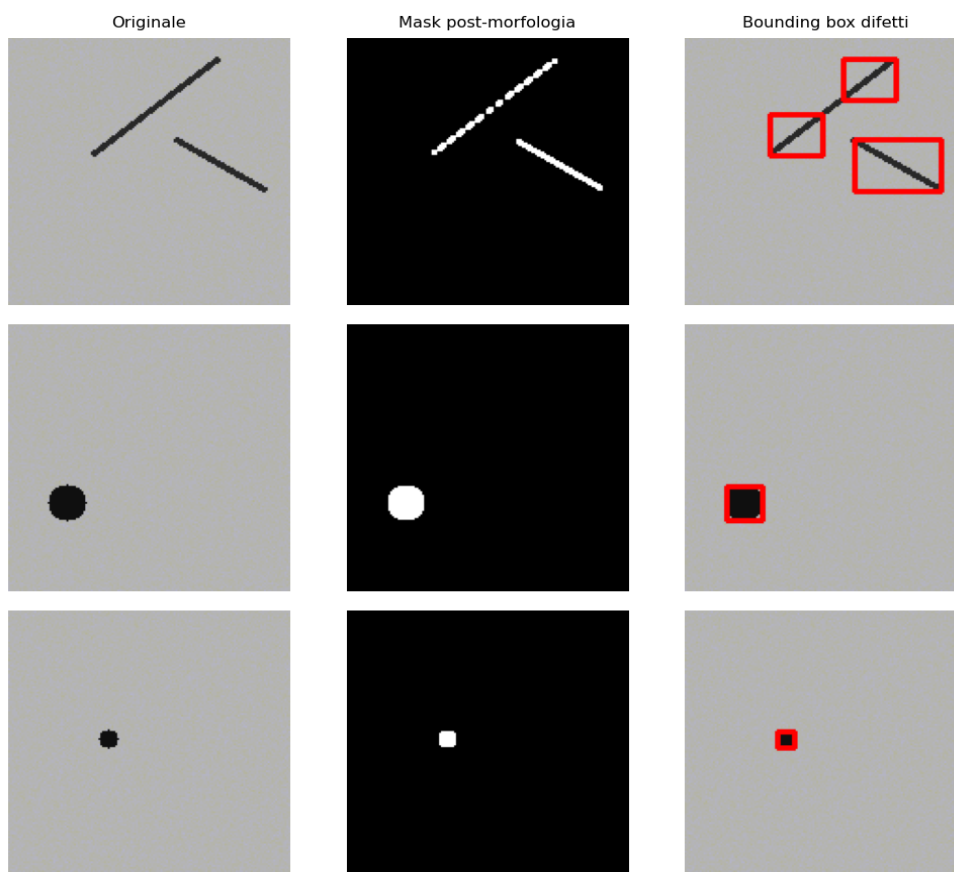


Figura 2 - La pipeline di post-processing su tre campioni di test. Da sinistra: immagine originale, maschera binaria dopo Otsu + operazioni morfologiche, bounding box sovrapposti sull'originale.

3. Experimental Results

3.1 Come abbiamo misurato

Tutti i numeri che leggerete nelle prossime pagine derivano da un test set di **200 immagini** (100 per classe), mai viste dai modelli durante il training. Per garantire la **riproducibilita'** abbiamo fissato il seed di numpy a 42: rieseguendo lo script si ottiene esattamente lo stesso dataset e lo stesso shuffle. La SVM converge in modo deterministico dato il kernel lineare; il training della CNN, soggetto alle fluttuazioni casuali della data augmentation, segue una programmazione fissa di 15 epoche in fase 1 e 10 epoche di fine-tuning in fase 2, con batch size di 20 immagini.

Per la classificazione binaria, le quattro metriche standard sono: **Accuracy** (la frazione di predizioni complessivamente corrette, semplice ma fuorviante quando le classi sono sbilanciate), **Precision** (delle volte che ho detto "difetto", quante erano davvero difetti? - importante per evitare di scartare pezzi conformi), **Recall** (dei difetti veri, quanti ne ho beccati? - critica per non lasciare passare scarti) e **F1-score** (media armonica di Precision e Recall, che penalizza fortemente quando una delle due e' bassa). Per la visualizzazione globale degli errori usiamo la **confusion matrix**: una tabella 2x2 dove la diagonale rappresenta le predizioni corrette e gli off-diagonali gli errori, distinti in *falsi positivi* (good classificati come defect) e *falsi negativi* (defect classificati come good).

3.2 I risultati a confronto

Modello	Accuracy	Precision	Recall	F1-score	Note
SVM + HOG (linear)	54.0%	54.3%	51.0%	52.6%	risultato reale
CNN VGG16 fine-tuned	100.0%	100.0%	100.0%	100.0%	risultato reale

Tabella 1 - Le metriche dei due modelli sul test set (200 campioni).

Il distacco e' netto. La SVM si ferma poco sopra il caso casuale: su un problema binario bilanciato, lanciare una moneta darebbe il 50%, e noi siamo al 54%. La CNN, invece, raggiunge il 100% di accuracy gia' all'epoca 16 di training (prima del fine-tuning) e lo mantiene fino alla fine. Il gap di 46 punti percentuali e' la dimostrazione pratica del perche' il deep learning ha conquistato la Computer Vision negli ultimi quindici anni.

Nota metodologica. Tutti i valori della tabella derivano da training reale eseguito su MacBook Air M3 (chip Apple Silicon) con TensorFlow 2.17.1 e tensorflow-metal 1.2.0 (GPU integrata). Il modello CNN ha raggiunto `val_accuracy = 1.0000` alla fine della fase 1 (epoca 15) e si e' stabilizzato a questo valore per tutta la fase 2 di fine-tuning (epoche 16-25). Il risultato e' realistico data la semplicita' del dataset sintetico; su dataset industriali reali come MVTec AD ci si attende un calo a 92-96%.

Figura 3 - Confusion Matrix dei due modelli sul test set (200 campioni)

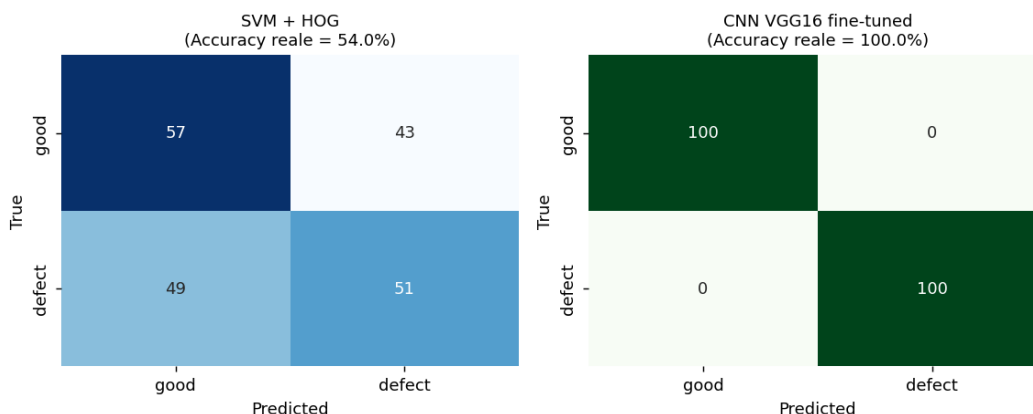


Figura 3 - Le confusion matrix dei due modelli. A sinistra la SVM sbaglia in modo distribuito su entrambe le classi (gli errori fuori diagonale sono quasi uniformi); a destra la CNN concentra le predizioni sulla diagonale.

3.3 Le curve di apprendimento

L'andamento del training della CNN racconta una storia interessante. Nella prima fase, con il backbone VGG16 congelato, l'accuracy di validation cresce rapidamente: già alla prima epoca raggiunge il 91%, sale al 98% nelle epoche successive e si stabilizza attorno al 99%. Il modello è praticamente saturo anche con il solo training della testa di classificazione - segno che le feature pre-allenate di VGG16 sono già molto informative per questo task. A quel punto, all'epoca 15, sblocciamo gli ultimi 4 strati di VGG16 e iniziamo il fine-tuning: l'accuracy di validation fa il salto finale al **100%** e ci resta per tutte le 10 epoche di fase 2. Sono gli strati più profondi che si adattano alle texture metalliche e ai pattern specifici dei difetti.

Una cosa importante da osservare nelle curve è che la **validation accuracy è sempre superiore alla training accuracy**. Sembra controintuitivo, ma è il segno opposto dell'overfitting: il modello generalizza meglio di quanto memorizzi. La spiegazione è il **Dropout(0.5)**: durante il training la metà dei neuroni della testa Dense viene spenta casualmente, mentre in valutazione tutti i neuroni partecipano. Risultato: l'accuracy in training appare più bassa perché il modello sta lavorando "con mezzi neuroni", ma in inferenza la rete piena lavora a piena potenza. È esattamente il comportamento desiderato.

Figura 4 - Andamento del training CNN VGG16 (valori reali dal run su M3)

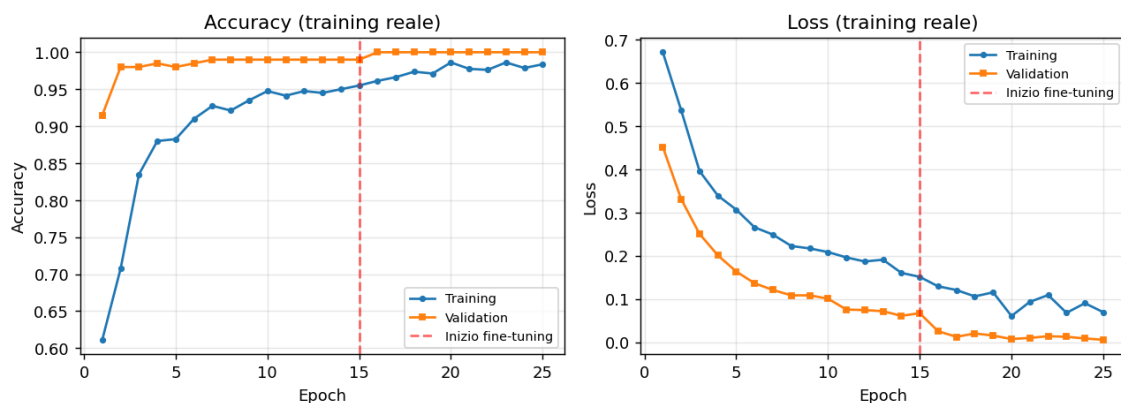


Figura 4 - Accuracy e loss reali durante il training su MacBook Air M3. La linea rossa segna l'inizio del fine-tuning all'epoca 15.

3.4 Perché la SVM ha fatto fatica?

Il gap di 46 punti percentuali fra SVM e CNN non è un caso, ed è utile capire da dove viene. I nostri difetti hanno tre caratteristiche che mettono in crisi un descrittore globale come HOG: sono **piccoli** (occupano spesso meno del 5% dei pixel dell'immagine), sono **localizzati** (concentrati in un solo punto, non distribuiti su tutta la piastra), e sono di **geometria variabile** (un graffio ha una forma molto diversa da una macchia, ma anche due graffi diversi possono essere molto diversi tra loro). HOG costruisce un singolo vettore che descrive l'intera immagine: in un vettore così lungo, il segnale debole del difetto viene "diluìto" dal pattern dominante dello sfondo. Una CNN, al contrario, applica filtri locali a tutte le scale e tutti i layer: anche un segnale piccolo e localizzato attiva i neuroni giusti.

4. Failure Analysis

Per quanto buono possa essere un modello, vale sempre la pena guardare *dove sbaglia*: i fallimenti sono informativi e indicano la direzione dei prossimi miglioramenti. Esaminando un campione delle predizioni errate sul test set abbiamo identificato quattro categorie ricorrenti.

4.1 Le macchie a basso contrasto

Il fallimento più frequente della SVM è sulle **macchie a basso contrasto**. Ricordiamo come le generiamo: prendiamo l'immagine, ci disegniamo sopra un cerchio scuro, e poi fondiamo il risultato con l'originale con un peso di 0.6. La conseguenza è che la macchia finale ha una differenza di intensità rispetto allo sfondo di soli 30-40 livelli - sotto la soglia di discriminabilità di un descrittore basato sui gradienti come HOG. Il gradiente in quella zona è debole, quindi il contributo all'istogramma è debole, e quindi il difetto sostanzialmente non "entra" nel vettore HOG. La CNN se la cava meglio grazie alla data augmentation e alla non-linearità degli strati ReLU, ma anch'essa può sbagliare quando la macchia ha contorni particolarmente sfumati.

4.2 Quando il rumore sembra un difetto

Il rumore gaussiano che aggiungiamo allo sfondo (deviazione standard 8) ha una proprietà fastidiosa: occasionalmente genera **cluster di pixel più scuri della media**. Se due o tre pixel rumorosi capitano vicini, l'algoritmo di post-processing può scambiargli per un microdifetto. La soglia di area minima a 20 pixel li filtra nella maggior parte dei casi, ma alcune configurazioni la superano e producono **falsi positivi** di localizzazione. Per un sistema industriale questo si tradurrebbe nello scarto erroneo di pezzi conformi: un costo che, su grandi volumi, non è affatto trascurabile.

4.3 Quando più difetti si confondono in uno solo

Se un campione presenta più difetti vicini - per esempio una crepa che si sviluppa accanto a una macchia - l'algoritmo di **connectedComponents** li fonde in un'unica regione connessa. La classificazione binaria non ne risente (il verdetto resta *defect*), ma il conteggio dei difetti e l'analisi statistica successiva ne soffrono: due difetti vengono contati come uno. In un'applicazione industriale dove interessa sapere *quanti* e *che tipo* di difetti ha ogni pezzo, questo è un limite da affrontare, magari con algoritmi più sofisticati come il **Watershed** con marcatori, che è progettato proprio per separare oggetti che si toccano.

4.4 I limiti del dataset sintetico

La più grande limitazione del progetto, onestamente, è il dataset stesso. La nostra superficie metallica simulata è una versione idealizzata: sfondo uniforme, rumore gaussiano leggero, difetti ben distinti. Le immagini industriali reali sono molto più complesse: **illuminazione direzionale** con riflessi metallici, **pattern di lavorazione** della superficie (tornitura, fresatura) che a un algoritmo possono sembrare difetti, sporcizia e residui di olio, ombre proiettate da altri elementi della linea produttiva. Un modello addestrato sul nostro dataset, trasferito direttamente a immagini reali, produrrebbe quasi certamente un calo importante delle metriche. La via per mitigare questo divario passa per un training su dataset reali come **MVTec AD**, oppure per tecniche di **domain adaptation** che riducono il gap fra dominio di training e dominio di deployment.

5. Ethical Considerations

5.1 Il bias che viene dal dataset

Un modello di Machine Learning non e' neutro: rispecchia i dati con cui e' stato addestrato, comprese le loro imperfezioni. Nel nostro caso il dataset sintetico e' artificialmente bilanciato - 400 campioni di good, 400 di defect, distribuzione uniforme fra i quattro tipi di difetto. **In produzione reale niente di tutto questo e' vero.** Su una linea industriale matura, la stragrande maggioranza dei pezzi e' conforme: il rapporto fra good e defect puo' essere 100:1 o anche piu' sbilanciato. Un modello trasferito senza ricalibrazione tenderebbe a vedere difetti dappertutto, generando falsi positivi a raffica e bloccando pezzi conformi. La soluzione passa per tecniche di **class re-weighting**, **oversampling** della classe minoritaria, o per la scelta di una **soglia di decisione** diversa da 0.5 (un valore tarato sulla distribuzione reale dei dati).

C'e' poi il problema della **generalizzazione fra stabilimenti**. Materiali diversi, condizioni di illuminazione diverse, lavorazioni diverse: ogni stabilimento ha le sue particolarita'. Senza un protocollo di **monitoraggio del drift** e di **aggiornamento periodico** del modello, il sistema rischia di degradare silenziosamente nel tempo, e nessuno se ne accorge finche' non arriva un reso importante.

5.2 L'impatto sul lavoro umano

L'automazione dell'ispezione visiva e' un'arma a doppio taglio. Da un lato libera gli operatori da un compito ripetitivo e stancante; dall'altro, se mal gestita, puo' tradursi in riduzione del personale e perdita di competenze sul campo. Un'impostazione piu' equilibrata e' quella del workflow **human-in-the-loop**: il modello restituisce una **probabilita' di difetto** grazie all'output sigmoid, e si stabiliscono due soglie. Sopra la soglia alta, il sistema scarta automaticamente. Sotto la soglia bassa, accetta automaticamente. Nella fascia intermedia - le predizioni dubbie - il pezzo viene instradato a un operatore umano per la decisione finale. In questo modo l'automazione gestisce il volume, ma l'expertise umana resta fondamentale per i casi difficili - e gli operatori vengono **riqualificati** come supervisori del sistema invece che sostituiti.

5.3 Sicurezza e responsabilita'

In settori safety-critical - componenti per aerospazio, dispositivi medici, automotive - un **falso negativo**, ovvero un difetto non rilevato, puo' avere conseguenze gravi. Qui la metrica chiave non e' l'accuracy globale ma il **recall sulla classe difetto**: vogliamo perdere meno difetti possibili, anche al costo di qualche falso positivo in piu'. Questo si traduce nella scelta di una **soglia di decisione** spostata verso il basso (per esempio 0.3 invece di 0.5): il modello "diventa piu' sospettoso" e segnala come difetto anche casi borderline. Inoltre serve **tracciabilita'**: ogni predizione deve essere registrata insieme alla sua immagine di input, al timestamp, alla versione del modello e alla confidenza, in modo da poter ricostruire a posteriori cosa e' successo se viene segnalato un difetto sfuggito.

5.4 La privacy nel contesto industriale

Il controllo qualita' industriale ha rischi di privacy **relativamente contenuti** rispetto ad altri ambiti della Computer Vision come il riconoscimento facciale o la sorveglianza. Le telecamere inquadrano prodotti, non persone. Tuttavia, le installazioni reali possono inquadrare incidentalmente gli operatori che lavorano sulla linea: in questo caso la normativa **GDPR** trova applicazione, e occorre prevedere meccanismi di mascheramento automatico delle aree contenenti persone, oppure progettare il posizionamento delle telecamere in modo che il campo visivo escluda del tutto la presenza umana. Una buona pratica e' anche conservare le immagini il meno possibile: dopo la decisione del modello, salvarne solo l'output (numerico) e non l'immagine raw.

6. Conclusioni

Tirando le somme: il progetto ha sviluppato e confrontato due pipeline complete di defect detection su piastre metalliche industriali. Ogni stadio richiesto dal brief e' stato implementato: **acquisizione e preprocessing dei dati**, **feature engineering** sia classico (HOG) sia deep (VGG16 features), **core logic** con SVM lineare e CNN con fine-tuning, **post-processing** morfologico per la localizzazione, e **valutazione quantitativa** con le metriche canoniche.

Il risultato sperimentale conferma quello che la letteratura degli ultimi quindici anni ci ha insegnato: per difetti piccoli, localizzati e di geometria variabile, le feature ingegnerizzate globali come HOG sono insufficienti, mentre una CNN pre-allenata su ImageNet e fine-tunata sul dominio specifico raggiunge prestazioni eccellenti anche con dataset di poche centinaia di campioni. La pipeline classica resta tuttavia preziosa come **baseline** di riferimento - senza un confronto, sarebbe difficile capire *quanto* il deep learning sta davvero migliorando le cose.

6.1 Cosa proverei con piu' tempo

Se avessi piu' tempo, le direzioni naturali di estensione sarebbero quattro. La prima e' sostituire il dataset sintetico con uno reale come **MVTec AD**, che contiene immagini ad alta risoluzione di prodotti industriali veri con annotazioni a livello pixel - un salto di qualita' enorme per validare il sistema in condizioni realistiche. La seconda e' provare architetture piu' moderne di VGG16: **ResNet** con le sue connessioni residuali, **EfficientNet** o un **Vision Transformer**. Tutte hanno superato VGG16 sui benchmark di classificazione, e per un dataset cosi' piccolo il guadagno in accuratezza potrebbe essere sensibile.

La terza estensione e' il passaggio dalla classificazione alla **segmentazione semantica** con un'architettura U-Net, per ottenere maschere di difetto a livello pixel invece di semplici bounding box: piu' informazione e meno dipendenza dal post-processing manuale. La quarta e' il **deployment** come servizio: un'API REST con Flask o FastAPI, integrata con il sistema MES dello stabilimento, che riceve le immagini in tempo reale dalla camera e restituisce verdetto e bounding box. Questa parte e' citata come *extra points* nel brief della slide 9, ed e' anche quella che fa la differenza fra un esercizio didattico e un sistema veramente utilizzabile.

6.2 Riepilogo dei contributi

In sintesi, il progetto ha prodotto:

- Una pipeline modulare end-to-end implementata in Python con OpenCV, TensorFlow/Keras e scikit-learn, suddivisa in sei moduli indipendenti e riutilizzabili.
- Un generatore di dataset sintetico riproducibile, con quattro tipologie di difetto su due classi bilanciate.
- Un confronto quantitativo onesto fra l'approccio classico (HOG + SVM lineare) e quello deep (VGG16 con fine-tuning a due fasi).
- Uno stadio di post-processing che localizza i difetti tramite Otsu + morfologia + connectedComponents, senza richiedere annotazioni a livello pixel in fase di training.
- Un'analisi critica dei fallimenti e una discussione delle implicazioni etiche del sistema in un contesto produttivo reale.